Day 09

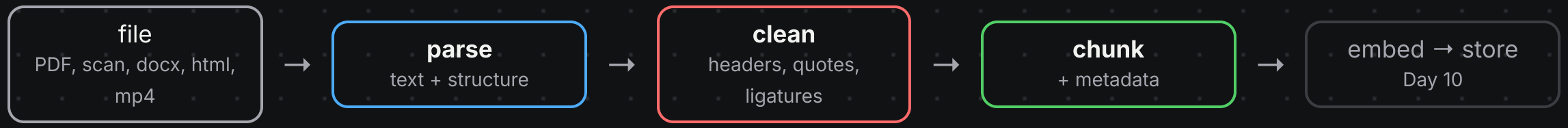
Parsing and chunking

RAG is only as good as what you feed it. Today: get clean text out of **any file**, then cut it into pieces that **keep their meaning**. On real Indian law, for our project **Kanoon.ai**.

1 Inside a PDF · 2 Coordinates · 3 Tables · 4 OCR · 5 Word & web · 6 Audio & video · 7 Cleaning · 8 Fixed & recursive · 9 Structure-aware · 10 Semantic & parent–child · 11 Contextual retrieval

garbage in, garbage out

Ingestion decides what RAG can ever find



What goes wrong here

- a scanned page gives **no text**, so it is never found
- page headers repeat in **every chunk**
- a section title ends up on the **wrong section**
- a chunk cuts the answer **in half**

Kanoon.ai

- ask about Indian law, get the **exact section** cited
- first Act: **Bharatiya Nyaya Sanhita, 2023** (replaced the IPC on 1 July 2024)
- 358 sections, 102 pages, Gazette of India PDF

1 · what's inside a PDF

Text layer, scan, or a text layer that lies

Text layer

- characters are stored
- `page.get_text()` works
- BNS pages: **`3,500–4,600`** chars each

Scan

- one picture per page
- **`0`** chars, **`1`** image
- needs **OCR**

Text layer that lies

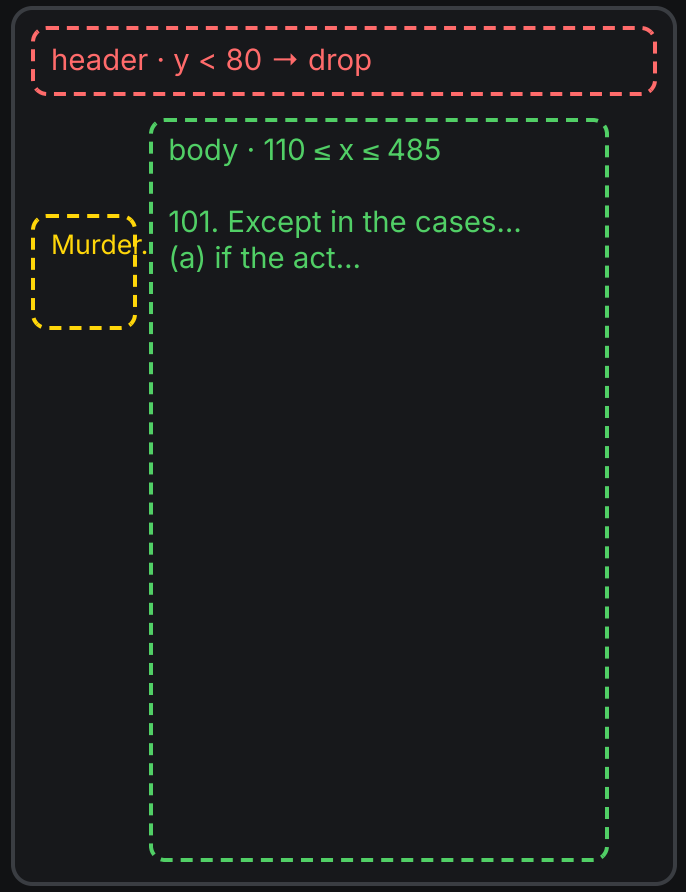
- Gazette masthead gives `vlk/kkj.k`
- it is **असाधारण** in an old Hindi font
- text exists, but it's garbage

```
# check every page, real files mix all three
for i, page in enumerate(doc):
    print(i + 1, len(page.get_text()), len(page.get_images()))
```

rule: having a text layer doesn't mean the text is right. **Look at the output** before you index it.

2 · PyMuPDF · layout

Use coordinates: header, body, margin



a Gazette page: 595 × 842 points

```
page.get_text("blocks")   # (x0, y0, x1, y1, text)
 58  178   84  'Murder.'           # margin
142  177  448  '101. Except in…'   # body
122   62  473  '32 THE GAZETTE…'   # header

BODY = pymupdf.Rect(110, 80, 485, 842)
page.get_text(clip=BODY)
```

- Plain `get_text()` puts `Murder.` at the **end of the page**, far from section 101
- Kanoon.ai: the title is the margin note **at the same height** as the section's first line
- `pymupdf4llm.to_markdown(header=False, footer=False)`: Markdown, headers dropped, OCR when needed

3 · the toolbox

Which parser when

document	first choice	what we saw today
digital PDF, simple layout	**PyMuPDF** / pymupdf4llm	fast, coordinates for your own rules
PDF with tables	**pdfplumber**	ruled IPC → BNS table came back as exact rows and cells
complex layout	**Docling** (IBM)	clean Word table, but moved `Murder.` to the top of the BNS page
Word, Excel, PowerPoint	**MarkItDown** or Docling	both kept headings, list and table
web page	keep `<main>`, then MarkItDown	nav, ad, sidebar and footer removed
scan	Tesseract / RapidOCR → Mistral OCR	OCR only pages without text
audio, video	ffmpeg → **Whisper**	keep timestamps as metadata

lesson: a heavier tool isn't always better. For one regular layout, **a few coordinate rules beat a general layout model**.

4 · OCR, measured

Score OCR against text you already know

Our scan is page 3 of the BNS sample, so we have the **true text** and can score each engine instead of guessing.

engine	word similarity	cost
Tesseract	0.967	free, local
RapidOCR (pymupdf4llm)	0.972	free, local
Docling	0.971	free, local
Mistral OCR	run with key	paid, per 1,000 pages

```
Tesseract mistakes:
likely → fikely
A is → Ais
A kills → Akills
Sec. 1] → See. 1]
```

rules: OCR **only** pages with no (or broken) text · keep the **page number** · paid OCR / vision LLMs are for handwriting, stamps, bad photos, complex tables

5–6 · web pages, audio, video

Boilerplate out, timestamps in

Web page

MarkItDown on the whole page kept the nav, *"50% off… LAW50"*, popular posts and cookie footer.

```
main = soup.find("main") or soup.find("article")
markitdown.convert_stream(main_html)
```

Boilerplate repeats on every page, so every chunk looks the same to search.

Video → audio → text

```
ffmpeg -i in.mp4 -vn -ac 1 -ar 16000 out.wav
```

said	Whisper base.en heard
Kanoon.ai	khanum.ai
liable to fine	liable to find
place of birth	face of birth

`initial_prompt` with the vocabulary fixed all of them, but merged 7 segments into 1. **Check every time.**

7 · cleaning

Small characters, big search problems

problem	example	fix
two apostrophes	`offender’s` ≠ `offender's`	map curly → straight
ligatures	`Offence` ≠ `Offence`	Unicode NFKC
running headers	`32 THE GAZETTE OF…`	regex / crop
hyphenation	`pun-\nishment`	join across lines
spacing	`An  Act  to`	collapse spaces

The text layer had `offender’s`, Tesseract returned `offender's`: a keyword search for one misses the other.

```
QUOTES = str.maketrans({"’": "'", "‘": "'",
                        "“": '"', "”": '"'})

def clean(text):
    text = unicodedata.normalize("NFKC", text)
    text = text.translate(QUOTES)
    text = HEADER_RE.sub("", text)
    text = re.sub(r"(\w)-\n(\w)", r"\1\2", text)
    text = re.sub(r"[ \t]+", " ", text)
    return text.strip()
```

rule: write cleaning rules **after looking at your parser's output**. Keep them small and tested.

8 · why chunk · fixed size

Fixed-size chunks cut answers in half

Precision: one vector for 358 sections is a blur. Small chunks match specific questions.

Cost: send the top few chunks, not the whole Act (Day 08: 3,300 vs 10,600 tokens).

Citations: a chunk knows it is "BNS Section 103, page 34".

test: is **Section 103 (Punishment for murder)** whole in one chunk?

500 chars, no overlap	28 chunks	**split**
500 chars, 100 overlap	35 chunks	**split**
500 chars, 250 overlap	56 chunks	**split**
200 tokens, 30 overlap	19 chunks	whole, by luck

```
same 200-token chunker, text starting at char:
0 whole · 100 split · 200 split
300 split · 400 whole · 500 whole
```

Overlap (10–20% is typical) only helps by chance, and adds duplicates. Count in **tokens**: models do.

9 · recursive · structure-aware

Split where the document says a unit starts

Recursive

Try the biggest break first: `"\n\n"` → `"\n"` → `". "` → `" "`. Chunks end at natural breaks.

LangChain `RecursiveCharacterTextSplitter`, the common default. Still doesn't know what a *section* is.

Structure-aware

Law has units: Chapter → Section → sub-section. A section starts with `103.` at the start of a line.

Accept only the next number, so a wrapped line starting `1860.` can't start a fake section.

```
if match and int(match[1]) == expected:      # 101, 102, 103 … in order
    sections.append({"section": expected, "text": line}); expected += 1
```

11 sections → **11 chunks**, every one whole. Section 101 has 1,492 tokens (5 exceptions, many illustrations), so **split long ones again** and keep the same title. Markdown from Docling or OCR? `MarkdownHeaderTextSplitter` does the same with # headings.

10 · semantic · parent–child

When there's no structure, and when small isn't enough

Semantic chunking

- embed every sentence (local `bge-small-en-v1.5`)
- a drop in similarity to the next sentence = new chunk
- ours: 15 chunks, 3 to 582 tokens; four were just `Illustrations.`
- good for transcripts and emails; **structure wins** when it exists

Parent–child (small-to-big)

- **children**: one sub-clause or illustration, embedded and searched
- **parent**: the whole section, sent to the LLM
- ours: 11 parents, 52 children
- search finds "(2) When a group of five or more persons...", the LLM gets all of Section 103

the tension: small chunks **search** better, big chunks **answer** better. Parent–child gives you both.

11 · metadata · contextual retrieval

Every chunk should say where it lives

On its own this chunk never says *murder*, *provocation* or *BNS*:

```
(b) given by anything done in obedience to the law, or
by a public servant in the lawful exercise of the powers
of such public servant;
```

Free: prepend the breadcrumb the parser already knows

```
Bharatiya Nyaya Sanhita, 2023 > Chapter VI > Section
101: Murder
(b) given by anything done in obedience…
```

LLM: Anthropic's prompt, "give a short succinct context to situate this chunk within the overall document". One call per chunk.

Anthropic, failed retrievals (1 – recall@20)	
contextual embeddings	-35%
+ contextual BM25 (Day 11)	-49%
+ reranking (Day 11)	-67%

context ≈ 50–100 tokens · ≈ $1.02 per 1M document tokens with prompt caching

Metadata: Act, section, title, page, source URL, `as_of` date, and a **stable `chunk_id`** (`bns :: s103 :: 0`) so re-ingesting replaces instead of duplicating.

side by side

Six strategies, same 3,127 tokens of BNS

strategy	chunks	avg tokens	max tokens	Section 103 whole?
fixed 500 chars	28	113	131	**no**
fixed 500 chars, 100 overlap	35	112	131	**no**
fixed 200 tokens, 30 overlap	19	193	200	yes, by luck
recursive 500 chars	32	98	122	yes
semantic	15	209	582	yes
structure (sections)	11	274	1,492	**yes, by design**

Small (100–200 tokens): precise search, may miss context, many vectors.

Large (800–1,000): has context, blurry search, fewer vectors.

No best size: measure on your own questions (homework, then Day 13).

wrap up

Kanoon.ai today, recap and homework

kanoon-ai/

```
python ingest.py bns
download | bns: 1350 KB
chunk | bns: 358 sections → 377 chunks
```

- `parse.py`: body column, margin titles, chapters, section numbers in order
- `chunk.py`: one chunk per section, > 800 tokens split again, breadcrumb

Homework

- chunk the full BNS: **fixed 500 tokens** vs **sections**
- write **10 questions** + the section that answers each
- count how often one chunk holds the whole answer
- bonus: add the BNSS or the Constitution. What breaks in `parse.py`?

Sources, checked 2026-10-23: BNS Gazette copy: prsindia.org (Act No. 45 of 2023), official page indiacode.nic.in/handle/123456789/20062 · pymupdf4llm: pymupdf.readthedocs.io/en/latest/pymupdf4llm · Docling: github.com/docling-project/docling · MarkItDown: github.com/microsoft/markitdown · faster-whisper: github.com/SYSTRAN/faster-whisper · contextual retrieval: anthropic.com/news/contextual-retrieval

All numbers from `day-09/code` and `kanoon-ai`, run 2026-10-23. Not legal advice.